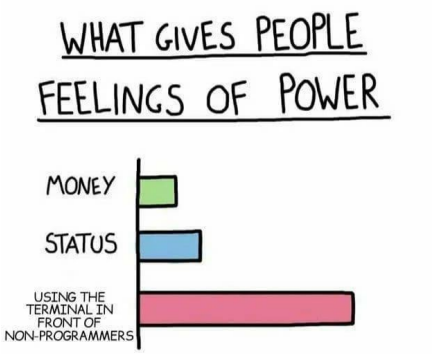
The Command Line

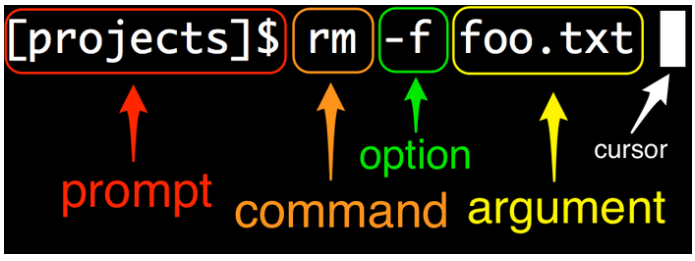
- Core Module
- Text interface to your computer
- Essential for MLOps and cloud interaction



1

Command Line Anatomy

- **Prompt**: Name of current directory followed by sign ($, >, etc.)
- **Command**: Actual command to execute (e.g., `ls`, `cd`)
- **Options**: Additional arguments for the command (e.g., `ls -l`)
- **Arguments**: Parameters passed to the command (e.g., `ls -l figures`)



2

Exercises

- Open a terminal and navigate using `cd` and `pwd`.
- Use `ls` with options like `-l`.
- Try commands: `which`, `echo`, `cat`, `wget`, `less`, `top`.
- Write and execute a Python script using `nano`.

3

Windows Users

- highly recommend installing Windows Subsystem for Linux (WSL).
- Use an elevated (admin) Windows Command Prompt.
- If not using WSL, work in Windows Command Prompt, not Powershell.

4

Bash Scripting

```
#!/bin/bash
# A sample Bash script
echo Hello World!
```

- Modify to call your Python script.
- Write a loop to execute the script multiple times.

Environment Variables

- **Windows:**

```
set MY_VAR=hello
echo %MY_VAR%
```

- **Linux/Mac:**

```
export MY_VAR=hello
echo $MY_VAR
```

- Use `os.environ` in Python to access variables.

Exercise

- Identify command, options, and arguments:

```
gcloud compute instances create-with-container instance-1 \
--container-image=gcr.io/<project-id>/gcp_vm_tester
--zone=europe-west1-b
```

- The command is `gcloud compute instances create-with-container`.
- The options are `--container-image=gcr.io/<project-id>/gcp_vm_tester` and `--zone=europe-west1-b`.
- The arguments are `instance-1`.
- The tricky part of this example is that commands can have subcommands, which are also commands. In this case, ‘compute‘ is a subcommand to ‘gcloud‘, ‘instances‘ is a subcommand to ‘compute‘, and ‘create-with-container‘ is a subcommand to ‘instances‘.

7

Exercise

- What do `-h` and `-V` options do?
- The `-h` (or `--help`) option prints the help message for the command, including subcommands and arguments. Try it out by executing `python -h`.
- The `-V` (or `--version`) option prints the version of the installed program. Try it out by executing `python --version`.

8

Exercise

- Practice and get comfortable with the command line.
- Watch this video for more.

https://youtu.be/oxuRxtrO2Ag

Importance of Package Managers

- Python has a vast ecosystem of packages.
- Need for package managers to install third-party packages.
- Popular package manager: `pip`.
- Missing feature in `pip`: Virtual environments.

Virtual Environments

- Isolate dependencies for different projects.
- Avoid version conflicts between projects.
- Example: Project A requires `torch==1.3.0`, Project B requires `torch==2.0`.
- Without virtual environments, dependencies will conflict.

```
cd project_A  # move to project A
pip install torch==1.3.0  # install old torch version
cd ../project_B  # move to project B
pip install torch==2.0  # install new torch version
cd ../project_A  # move back to project A
python main.py  # try executing main script from project A
```


Creating Virtual Environments

```
cd project_A
python -m venv env
source env/bin/activate
pip install torch==1.3.0
cd ../project_B
python -m venv env
source env/bin/activate
pip install torch==2.0
```

Creating Virtual Environments (Windows)

```
cd project_A
python -m venv env
.\env\Scripts\activate
pip install torch==1.3.0
cd ../project_B
python -m venv env
.\env\Scripts\activate
pip install torch==2.0
```


13

Popular Package Managers

- conda
- pipenv
- poetry
- pipx
- hatch
- pdm

14

Recommendation

- Use `conda` for creating environments.
- Use `pip` for installing packages.

Dependencies Management

- Use `requirements.txt` to specify dependencies.
- Specify package version numbers for reproducibility.
- Avoid dependency conflicts with correct versioning.

Example: Specifying Dependencies

```
package1            # any version
package2 == x.y.z   # exact version
package3 >= x.y.z   # at least version x.y.z
package4 >  x.y.z   # newer than version x.y.z
package5 <= x.y.z   # at most version x.y.z
package6 <  x.y.z   # older than version x.y.z
package7 ~= x.y.z   # version >= x.y.z and < x.(y+1)
```

- The version number is split into three parts: `x.y.z` where
  - `x` is the major version,
  - `y` is the minor version and
  - `z` is the patch version.

Dependency Resolution

- Process of determining compatible packages.
- Non-trivial problem; requires advanced algorithms.
- Example of conflicting dependencies:

```
pip install "matplotlib >= 3.8.0" "numpy <= 1.19" --dry-run
```

- need to relax the constraints

```
pip install "matplotlib >= 3.8.0" "numpy <= 1.21" --dry-run
```

Exercise 1

1. Install `conda` or `miniconda`.
2. Verify installation with `conda help`.

See Video for more.

https://youtu.be/MUZtVEDKXsk

Exercise 2

1. Create a virtual environment `my_environment` with Python 3.11.
2. List all created environments with `conda env list`.

Exercise 3

1. Export environment to `environment.yaml`.
2. Create a new environment from `environment.yaml`.

Exercise 4

1. Use `pip` to list installed packages and export to `requirements.txt`.

Exercise 5

1. Install `pipreqs`.
2. Use `pipreqs` to create a minimal `requirements.txt`.

Exercise

1. Install `pytest < 4.6` and `pytest-cov==2.12.1`.
2. Resolve dependency conflict by updating `pytest` version.

Editor/IDE Overview

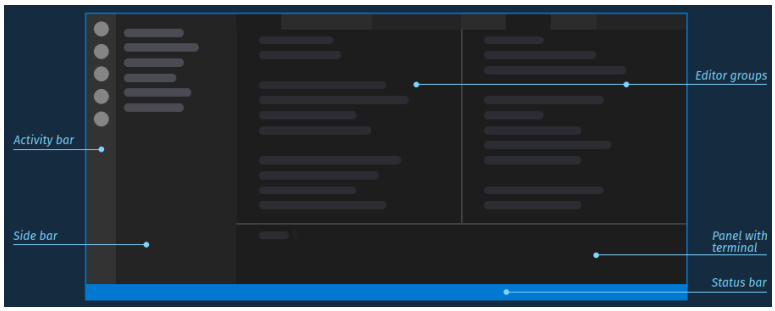
- Notebooks are useful for testing and visualization.
- For larger projects, use multiple `.py` files.
- A good editor/IDE is essential for productivity.

Recommended Editors

- **Spyder**
  - https://www.spyder-ide.org/
  - Matlab-like environment, easy to get started with.
- **Visual Studio Code**
  - https://code.visualstudio.com/
  - Multi-language support, fairly easy setup.
- **PyCharm**
  - https://www.jetbrains.com/pycharm/
  - IDE for Python professionals, steeper learning curve.

VS Code Interface

- **Action bar**: Navigate between different applications and extensions.
- **Sidebar**: File explorer and extension-specific functionality.
- **Editor**: Main code area, supports custom layouts.
- **Panel**: Terminal, environment management, etc.
- **Status bar**: Information on extensions and environment.



Exercises

- Create a new file.
- Run a Python script.
- Change the Python environment.

VS Code Setup

1. Install the `Python` extension for VS Code.
2. Install `Pylance` for enhanced code completion.
3. Install the `Jupyter` extension for notebook support.
4. Install `Python Environment Manager` for virtual environment management.

29

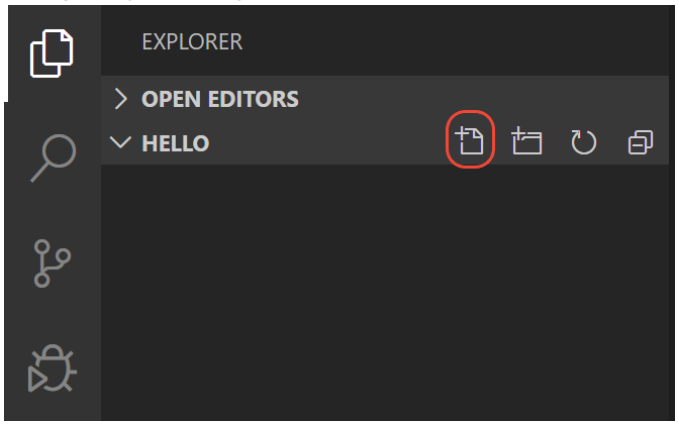
Changing Python Environment

- Click the Python version in the status bar.
- Select the desired environment from the list.

30

Project Navigation

- Create a folder `hello` and open it in VS Code.
- Create a new file `hello.py`.
- Use the Explorer to navigate your project.



Running Code

- Click the `run` button to execute the code.
- Use `Shift+Enter` to run selected code.
- Right-click to run in an interactive window.

Jupyter Notebooks in Production

- Jupyter is great for development but can be challenging for deployment.
- `nbconvert` can convert notebooks to `.py` scripts.
- Use `pip install nbconvert` to install.

Converting Notebooks

Produces `my_notebook.py` script.

```
jupyter nbconvert --to=script my_notebook.ipynb
```

AI Assistance

- Context-switching reduces productivity.
- **GitHub Copilot** integrates AI directly into VS Code.
- Provides suggestions based on current code context.

GitHub Copilot Exercises

1. Sign up for the **Student Developer Pack**.
2. Install the **GitHub Copilot extension**.
3. Try writing Python code and get suggestions.

Example Code

```
import torch
from torch import nn
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(28*28, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 10)
    def forward(self, x):
        x = x.view(-1, 28*28)
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        return x
model = Net()
print(model(torch.randn(1, 1, 14, 14)))
```


Using Copilot for Assistance

- Use `Ctrl+i` to open chat and ask questions.
- Highlight code and use `Explain This (Terminal)` command.

Comparing Copilot and ChatGPT

- Copilot is tailored for coding assistance.
- ChatGPT is a general-purpose model.
- Evaluate both tools for different use cases.

Notes

- Experiment with different editors and tools.
- Use AI tools to enhance productivity but always critically evaluate suggestions.
- Practice with VS Code and GitHub Copilot to improve your coding workflow.

Importance of Code Organization

- Improves code readability and maintainability.
- Reduces the risk of creating a “Big Ball of Mud”:
- *“A haphazardly structured, sprawling, sloppy, duct-tape-and-baling-wire, spaghetti-code jungle.”* - Brian Foote and Joseph Yoder, 1997
- Essential for data science and machine learning projects where data plays a central role.

Cookiecutter

- Tool for creating projects from **project templates**.
- We use a custom MLOps template, based on the cookiecutter data science template.
- Provides a **standardized** way of organizing project structures.
- Facilitates better code understanding and collaboration.

```
├── LICENSE            <- Open-source license if one is chosen
├── Makefile           <- Makefile with convenience commands like `make data` or `make train`
├── README.md          <- The top-level README for developers using this project.
├── data
│   ├── processed      <- The final, canonical data sets for modeling.
│   └── raw            <- The original, immutable data dump.
│
├── docs               <- Build basic documentation
│
├── models             <- Trained and serialized models, model predictions, or model summaries
│
├── notebooks          <- Jupyter notebooks.
│
├── pyproject.toml     <- Project configuration file with package metadata for
│                         and configuration for tools like black
│
├── reports            <- Generated analysis as HTML, PDF, LaTeX, etc.
│   └── figures        <- Generated graphics and figures to be used in reporting
│
├── requirements.txt   <- The requirements file for reproducing the analysis environment
│
├── tests              <- Test files
│
└── {{cookiecutter.project_name}}  <- Source code for use in this project.
    │
    ├── __init__.py    <- Makes folder a Python module
    │
    ├── data           <- Scripts to download or generate data
    │   ├── __init__.py
    │   └── make_dataset.py
    │
    ├── models         <- model implementations, training script and prediction script
    │   ├── __init__.py
    │   ├── model.py
    │   ├── train_model.py
    │   └── predict_model.py <- script for training the model
    │
    └── visualization  <- Scripts to create exploratory and results oriented visualizations
        ├── __init__.py
        └── visualize.py
```

Using a Template

- Templates serve as a guide, not a strict rule.
- Adapt templates to suit specific project needs.
- Helps in organizing machine learning projects effectively.

Python Project Structure

- Common structure for a Python package:

```
├── src/
│   ├── __init__.py
│   ├── file1.py
│   ├── file2.py
├── pyproject.toml
```

pyproject.toml

- Standardized way of describing project metadata.
- Introduced in PEP 621.
- Example structure:

```
requires = ["setuptools", "wheel"]
build-backend = "setuptools.build_meta"
[project]
name = "my-package-name"
version = "0.1.0"
authors = [{name = "EM", email = "me@em.com"}]
requires-python = ">=3.8"
dynamic = ["dependencies"]
[tool.setuptools.dynamic]
dependencies = {file = ["requirements.txt"]}
```


Advantages of pyproject.toml

- Declarative format using TOML.
- Can specify metadata for multiple tools.
- Reduces need for multiple configuration files.

setup.py + setup.cfg

- Older method for project configuration.
- Example structure:

```
# setup.py
from setuptools import setup
setup(
    name="my-package-name",
    version="0.1.0",
    author="EM",
    description="Something cool here."
    install_requires=[
        'torch==2.1.0',
        'matplotlib>=3.8.1'
    ],
)
```


Installing Python Packages

- Basic installation command:

```
pip install .
```

- For developer mode:

```
pip install -e .
```

- `-e` stands for `--editable` mode, allowing immediate code updates.

Notes

- Effective code organization is crucial for maintainability.
- Cookiecutter helps standardize project structures.
- Python project configuration is moving towards `pyproject.toml`.

Exercises

- Exercises involve structuring the CNN MNIST classifier from previous exercises using `cookiecutter`.
- Perform tasks from the root directory.

```
python <project_name>/data/make_dataset.py
data/raw data/processed
python <project_name>/models/train_model.py
<arguments>
```

Exercise 1: Install Cookiecutter

- Install the cookiecutter framework.

```
pip install cookiecutter
```


Exercise 2: Create New Project

- Use the template.

```
cookiecutter <url-to-template>
```

- **Note:** Follow PEP8 guidelines for naming packages.

```
my_project (valid), MyProject (invalid).
```

Exercise 3: Create Virtual Environment

- Create and then install the project in a virtual environment.

```
pip install -e .
```


Exercise 4: Fill make_dataset.py

- Process raw data, normalize it, and save to `data/processed`.

```
# make_dataset.py
code for processing data
```

Exercise 5: Use Makefile

- Use predefined commands in the Makefile:

```
make data         # run make_dataset.py
make clean        # remove __pycache__
make requirements # install requirements.txt
```


Exercise 6: Organize Model Files

- Organize model files in the `models` folder.
- Save trained models and visualizations in appropriate folders.

Exercise 7: Add Train Command to Makefile

- Add a train command to Makefile.

```
train:
    python <project_name>/models/train_model.py
```


Exercise 8: Fill predict_model.py

- Create a file for model prediction.

```
python <project_name>/models/predict_model.py \
    models/my_trained_model.pt \
    data/example_images.npy
```

Exercise 9: Fill visualize.py

- Load pre-trained network and visualize intermediate features.
- Save visualizations to `reports/figures/`.

```
# visualize.py
Visualization code
```


Exercise 10: Update README and Requirements

- Update `README.md` with usage instructions.
- Update `requirements.txt` with necessary packages.

Steps to create and push a new GitHub repository

- 1. Create a new GitHub repo.
- 2. Run `cookiecutter` with the desired template.
- 3. Use the following commands:

```
cd <project_name>
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/<username>/<repo_name>
git push origin master
```


Notes

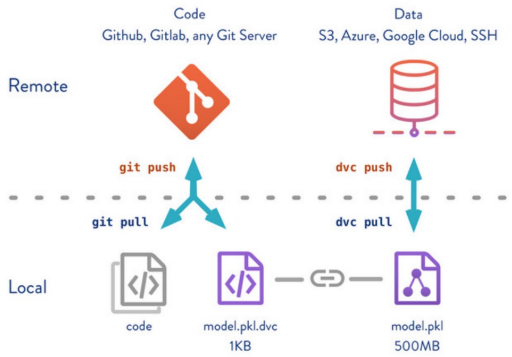
- Use `cookiecutter` to organize code.
- Update and maintain templates for team usage.
- Consider using `cruft` for template updates.

Data Version Control

- Focus on version control of data.
- Classic version control is for code files, data requires more space.
- Data size can exceed petabytes (1,000,000 GB).
- Use specialized frameworks for data versioning:
  - DVC
  - DAGsHub
  - Hub
  - Modelstore
  - ModelDB
- Version control the pointer to data instead of the data itself.

DVC: What is it?

- DVC extends `git` for versioning data, models, and experiments.
- Keeps track of small metafiles pointing to remote locations for data.
- Uses commands like `dvc pull/push` for data.
- Stores large data files in remote locations (e.g., Google Drive, `S3` bucket).



DVC Commands

- Initializes DVC in a repository.

```
dvc init
```

- Adding Remote Storage: Adds Google Drive as remote storage

```
dvc remote add -d storage gdrive://<your_identifier>
```

- Adding Data: Tracks data in the data/ directory

```
dvc add data/
```


Exercises

- Install DVC and the Google Drive extension.

```
pip install dvc
pip install dvc-gdrive
```

- Initialize DVC in your repository.

```
dvc init
```

- Add remote storage.

```
dvc remote add -d storage gdrive://<your_identifier>
```

- Add data files using DVC.

```
dvc add data/
```

- Commit and tag metafiles.

```
git add data.dvc .gitignore
git commit -m "First datasets"
git tag -a "v1.0" -m "data v1.0"
```

Updating Data

1. Add new data files to the repository.
2. Re-run data pipeline to incorporate new data.
3. Add, commit, and tag the metafiles.

```
dvc add
git add
git commit
git tag
dvc push
git push
```

4. Checkout to previous versions.

```
git checkout v1.0
dvc checkout
```


Large Dataset Handling

- Zip files into a single archive for version control.
- Use single file formats like `.parquet` or `.csv`.
- Version control the archive or single file.

Notes

How do you know that a repository is using DVC?

Check for `.dvc` directory or use `dvc status`.

Sequence of 5 commands for version control of a folder.

```
dvc add data/
git add .
git commit -m "added raw data"
git push
dvc push
```